AI ASSISTED CONTRACT REVIEW SYSTEM

Software Requirement Specification

James Mitchell Vore
AI Data Analytics
NemaLife Inc
Lubbock, Tx
jvore@ttu.edu

Ethan Benjamin Childs
Data Engineering
Paccar
Denten, Tx
Etchilds@ttu.edu

Nathan Cannon
Student
Texas Tech
Lubbock, TX
Nate.Cannon2004@gmail.com

Nolan Austin Jones
Student
Texas Tech
Lubbock, Tx
nolajone@ttu.edu

Tejas Sonni
Student
Texas Tech
Lubbock, Tx
tesoni@ttu.edu

FEBRUARY 24, 2026

SENIOR CAPSTONE PROJECT

Software Requirements Specification (SRS)
AI-Assisted Contract Review System

Abstract

This document presents the Software Requirements Specification (SRS) for the AI-Assisted Contract Review System. The proposed system leverages natural language processing (NLP), semantic embeddings, and vector-based information retrieval to assist users in analyzing legal contracts. Users will be able to upload contract documents, submit natural language questions, and receive responses grounded in the contract's contents. Additionally, the system will identify predefined high-risk clauses, commonly referred to as red flags, and provide structured explanations. The goal of this project is to demonstrate the practical application of modern AI techniques in document analysis while maintaining modular system architecture, clear limitations, and responsible AI usage.

1. Introduction

Contracts are foundational to business operations, employment agreements, vendor relationships, and intellectual property management. Despite their importance, contracts are often lengthy, complex, and written in dense legal language that can be difficult for non-experts to interpret. Individuals and small organizations frequently lack immediate access to legal professionals, making it challenging to quickly understand obligations, liabilities, termination conditions, and risk exposure.

Recent advancements in artificial intelligence, particularly in natural language processing, provide new opportunities for intelligent document analysis. Large language models and semantic embedding techniques allow text to be represented numerically in a way that captures meaning rather than simple keyword overlap. By combining semantic search with structured system design, it is possible to build assistive tools that help users navigate complex documents more efficiently.

The AI-Assisted Contract Review System aims to apply these technologies to contract analysis. The system is not intended to replace legal professionals or provide legal advice. Instead, it functions as an educational and analytical support tool that highlights relevant clauses and surfaces potential risks based on predefined criteria.

2. Overall Description

The system allows users to upload contracts in PDF, DOCX, or TXT format. After upload, the document is processed and segmented into structured clauses. Each clause is converted into a semantic embedding using a pre-trained embedding model. These embeddings are stored in a vector database to enable efficient similarity search.

When a user submits a question, the system converts the question into an embedding and retrieves the most semantically relevant contract clauses using cosine similarity. These retrieved clauses are returned through an API endpoint and can be passed to a question-answering module for response generation. Simultaneously, the system analyzes the contract for predefined red-flag patterns.

3. System Architecture

The system architecture consists of four primary components: (1) Document Ingestion and Preprocessing, (2) Semantic Retrieval Engine, (3) Question Answering Module, and (4) Red Flag Detection Engine. The architecture follows a modular design, allowing each component to operate independently while communicating through defined APIs.

The Semantic Retrieval Engine is the core component responsible for embedding generation, vector storage, and similarity search. It acts as the bridge between raw contract data and higher-level reasoning modules.

4. Functional Requirements

FR1: The system shall accept contract uploads in PDF, DOCX, and TXT formats.

FR2: The system shall extract and preprocess contract text into structured clauses.

FR3: The system shall generate semantic embeddings for each clause using a pre-trained model.

FR4: The system shall store embeddings in a vector database optimized for similarity search.

FR5: The system shall retrieve the top-k relevant clauses in response to user queries.

FR6: The system shall provide an API endpoint for retrieval operations.

FR7: The system shall identify predefined red-flag clauses and assign severity levels.

FR8: The system shall return structured JSON responses containing clause metadata and similarity scores.

5. Non-Functional Requirements

NFR1: The system shall maintain response times suitable for interactive use.

NFR2: The system shall be modular, maintainable, and extensible.

NFR3: The system shall clearly state limitations and disclaim legal advice.

NFR4: The system shall handle contracts of varying length without significant degradation in performance.

NFR5: The system shall ensure secure handling of uploaded documents during processing.

6. Impact and Usefulness

This project demonstrates the integration of AI technologies into practical software systems. By enabling semantic search and automated risk identification, the system improves accessibility to legal information and reduces the cognitive burden associated with contract review. While it does not replace professional legal advice, it serves as a powerful assistive tool for preliminary analysis and educational exploration.

7. Group Assignments and Schedule

The group will meet bi-weekly, once on Tuesday and once on Thursday, both after the scheduled class time. These meetings will be used to review progress, address technical challenges, ensure alignment between components, and verify that project milestones are being met. Additional meetings may be scheduled as needed prior to major deliverables.

Ethan Childs serves as the Project Lead and is responsible for ensuring compliance with assignment requirements and maintaining alignment with the Software Requirements Specification. He will oversee overall system integration, monitor progress toward deadlines, manage the group schedule, and ensure that all submission requirements are met on time.

Nolan Jones is responsible for front-end development and document preprocessing. He will design and implement the user interface that accepts contract uploads and user queries. Additionally, he will handle document parsing and segmentation, breaking down submitted contracts into structured chunks. These chunks will be formatted into a JSON-based structure suitable for processing by the embedding pipeline.

James Vore is responsible for building the semantic retrieval and embedding pipeline. He will select and integrate a pre-trained embedding model, convert contract chunks into semantic vector representations, and store those vectors in a vector database. He will implement similarity search functionality to retrieve the top-k most relevant clauses in response to user queries. Additionally, he will develop the API endpoint that exposes this retrieval functionality to other system components.

Nate Cannon is responsible for LLM integration and question-answering functionality. He will design and implement prompt templates that use retrieved contract clauses as context, integrate the large language model API, and ensure responses are grounded in the contract text. He will also implement guardrails to reduce hallucination and ensure that outputs clearly reference relevant contract sections.

Tejas Soni is responsible for the red-flag detection functionality. He will define a taxonomy of high-risk contract clauses, including but not limited to unilateral termination, uncapped liability, broad indemnification, and arbitration provisions. He will implement rule-based detection logic and, if feasible, integrate LLM-based classification to identify nuanced risk patterns. He will also assign severity levels and generate structured explanations for flagged clauses.

Together, the team's modular division of responsibilities ensures that each component can be developed independently while maintaining a cohesive and well-integrated overall system architecture.